Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

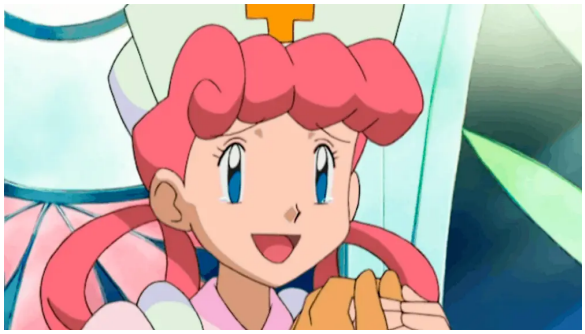
Taller 3

Objetivos

- Diseñar algoritmos para trabajar con Heaps
- Comprender en que casos es útil el uso de Heaps.

Introducción

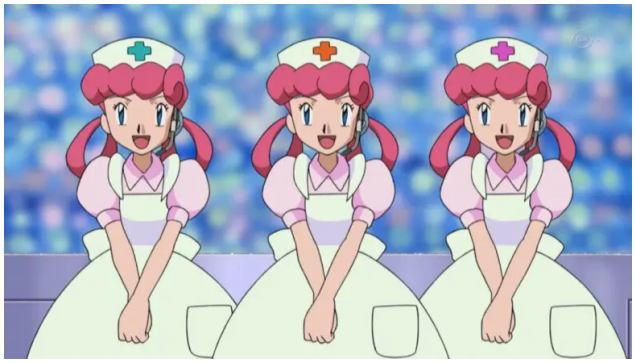
En pleno furor de la liga Pokémon, el Centro Pokémon parece una zona de combate más. Las puertas no dejan de abrirse, los entrenadores llegan corriendo de un lado a otro y la enferma Joy apenas alcanza a respirar entre tantos Pokémon agotados, heridos y fuera de combate. Decidir quién debe ser atendido primero ya no es solo cuestión de orden, sino de supervivencia.



Ante semejante caos, la enfermera Joy necesita tu ayuda: necesita un sistema eficiente que le permita organizar la atención de los Pokémon según la urgencia en cada caso.

Tu misión será diseñar una estructura capaz de manejar esta avalancha de emergencia con rapidez y precisión. Deberás implementar un sistema de cola de prioridades que permita al centro atender a los Pokémon sin morir en el intento.

¡Caos en el centro Pokémon!



Problema

Ante el caos en el centro Pokémon la enfermera Joy define el siguiente sistema para atender a los Pokémones: Existirán 2 tipos de prioridades, por el atributo `HP` y el atributo `Speed` en base a lo siguiente:

- Los Pokémon que tengan **MENOR** `HP` tendrán más prioridad.
- Los Pokémon que tengan **MAYOR** `Speed` tendŕan más prioridad.

Cada Pokémon puede ser atendido **más de una vez** pero **solo una vez** por prioridad. Es decir, esto define 2 colas de prioridad, una para cada atributo y cada Pokémon se encuentra en las 2 colas al llegar al centro Pokémon. Por ejemplo, el Pokémon 1 puede ser atendido ya que es el que tiene mayor prioridad en la cola de `HP`, lo que lo sacaría de esa cola pero aún se mantendría en la cola de `Speed`

Entidades

Pokemon: Cada pokemon cuenta con un `ID` único, un valor de `HP` y un valor de `Speed`. Todos los atributos son valores numéricos.

Flujo

Para esta parte, contarás con las funciones necesarias para hacer una inserción en un heap.

Primero recibirás un entero `M` que indica la cantidad máxima de pokemones que podrán entrar al centro pokemon en todo el flujo. Luego recibirás un entero `N` que indica la cantidad de eventos a recibir.

Parte Formativa - Eventos

`ENTER {pokemon_id} {HP} {speed}`

Este evento representa la entrada del pokemon con id `pokemon_id` cuyos atributos son `HP` y `speed` al centro pokemon.

Recuerda que al momento de llegar al centro pokemon, el pokemon debe entrar en las dos colas de prioridad.

El resultado esperado debe mostrar el siguiente output:

output

```
Centro Pokemon: nuevo pokemon id {pokemon_id}
```

Importante: Se requiere que esta operación tenga una complejidad no mayor a $\mathcal{O}(log(n))$ con n la cantidad de pokemones en las colas.

`EXTRACT {attr}`

Este evento extrae al pokemon más prioritario de la cola de prioridad `attr`.

Recuerden que el evento extraer elimina el pokemon de la cola que fue extraido pero no es eliminado de la otra cola.

El resultado esperado debe mostrar el siguiente output:

output

```
Centro Pokemon: atendido id {pokemon_id} segun prioridad {attr}
```

Importante: Se requiere que esta operación tenga una complejidad no mayor a $\mathcal{O}(log(n))$ con n la cantidad de pokemones en las colas.

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `centro` que se ejecuta con el siguiente comando:

```
./centro input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind deberás utilizar el siguiente comando:

```
valgrind ./centro input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

El archivo de input comenzará con el número `M` que indica la cantidad máxima de pokemones que entraran al centro pokemon, posterior a ello se entrega un número `N` que corresponde al número de eventos a recibir, y las `N` líneas correspondientes a cada uno.

Como ejemplo tenemos el siguiente input:

input

```
3 # Cantidad de magnezones
6 # Cantidad de eventos
ENTER 1 1 67
ENTER 2 5 23
ENTER 3 15 40
EXTRACT HP
EXTRACT Speed
EXTRACT Speed
```

Output

output

```
Centro Pokemon: nuevo pokemon id 1
Centro Pokemon: nuevo pokemon id 2
Centro Pokemon: nuevo pokemon id 3
Centro Pokemon: atendido id 1 segun prioridad HP
Centro Pokemon: atendido id 1 segun prioridad Speed
Centro Pokemon: atendido id 3 segun prioridad Speed
```

Evaluación (Solo parte evaluada)

La nota de tu taller es calculada a partir de testcases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán testcase publicos para que puedas probar tu implementación durante el taller.

Para el taller 1, no se evaluará que no tengan leaks y/o errores de memoria, pero se recomienda practicar como manejar correctamente la memoria, ya que se evaluará en talleres posteriores.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 3 segundos** y utilizar menos de 1 GB de RAM[1], de lo contrario recibirás 0 puntos en ese test. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

Integridad académica e IA

Cualquier acto deshonesto o fraude académico esta estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablet, teléfonos durante la evaluación.

Proceso de Entrega

Conexión a internet

Una vez iniciado sesión en el escritorio, seleccionar el icono de redes arriba a la derecha. "*availablenetwork*" → "*eduroam*". Para conectarse a eduroam, deben tener la siguiente configuración:

- Tener seleccionado la opción "No CA certificate is required".
- En la opción de "*Inner authentication*" seleccionar "*PAP*"
- Luego debe ingresar tus credenciales usuario uc. **IMPORTANTE:** en el usuario debes incluir el @, no solo el nombre.
- Ejemplos correctos: "*nombre@uc.cl*";"*nombre@estudiante.uc.cl*", Incorrecto: "nombre"sin el @.

Descargar taller

Para descargar el código base que con el que trabajaran, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisara al inicio de cada taller.

Editor de código

El editor de código de la imagen se llama "$lite - xl$". Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de código, buscar la sección de "$file$" y seleccionar "*Open*" y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar "*Openwith...*" y seleccionar el editor.

[1] Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`

- **IMPORTANTE:** El editor no tiene autoguardado, por lo que cada vez que hagan cambios deben usar "$CTRL + S$" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilara con la ultima versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR**.

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, verán una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan dentro de esa carpeta. Eso les asegura que en caso de que la imagen falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- src: carpeta con su código
- makefile: archivo para compilar
- tests: carpetas de tests publicas
- run.sh: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usan:

```
./run.sh
```

El run.sh genera una carpeta llamada outputs que guarda para cada test, el output de su programa. **Si desean ejecutar un único test** ocupan

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, puedes revisarlo en las carpetas dentro de src, ejemplo de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: tests/F1/pokemones/easy-0/input.txt y output.txt es un archivo txt que crean y es donde ira su salida.

En caso de querer debugear algún error de un test en especifico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, también es importante que estén conectados a internet, entrar en una terminal, posicionarse en la ruta donde esta la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde: "$nombre_carpeta$" es el nombre de la carpeta que quieren entregar, por ejemplo: "$T2_evaluado_G2$". "TX" donde X corresponde al taller que se esta entregando: por ejemplo: "T2". Y $nAlumno1_nAlumno2_...$ que son los números de alumnos de los integrantes del grupo que trabajaron separados por "_" (Si están realizando este taller de manera solitaria, solamente escriben su numero de alumno).
Ejemplo Correcto:

12345678_23456789 20387645 20387645_12345678_23456789

Ejemplo Incorrecto:

1234567823456789 $Nombre1$_12345678 $Nombre1_Nombre2_Nombre3$